Database Migration with Debezium — Cheat Sheet

Bookshop: Oracle AI Database 26ai Free (PDB FREEPDB1, schema BOOKSHOP) -> PostgreSQL 18 database bookshop

2026-11-09 | Debezium 3.7.1.Final | Kafka 4.x (KRaft)
Connectors: Java 17+ | Server, Operator: Java 21+
Kafka localhost:9092 | Connect REST localhost:8083

Strategy | Events | Source setup | Connectors and sink | Snapshots and signals | Transformations | Operations | 3.7 changes

1. Migration phases and the source of truth

Phase	Writes (source of truth)	Change stream	Rollback
1 Snapshot + streaming	Legacy Oracle	Oracle -> Kafka -> PG	Drop target tables, snapshot again
2 Parallel run + reconciliation	Legacy Oracle	Oracle -> Kafka -> PG	Fix translator, re-snapshot the table
3 Read cut-over	Legacy Oracle	Oracle -> Kafka -> PG	Route reads back to legacy
4 Write cut-over + reverse CDC	PostgreSQL	PG -> Kafka -> Oracle	Route writes back, re-enable forward CDC
5 Decommission	PostgreSQL	None	Restore from the archive

- Exactly one source of truth per table at any moment. One direction per table: a table captured both ways echoes every change.
- Reads cut over before writes, cohort by cohort, behind the strangler proxy. The point of no return is a recorded decision.
- Exit 1: snapshot done, lag stable at seconds. Exit 2: reconciliation clean for a full business cycle.

2. Database prerequisites

- Oracle: ARCHIVELOG with retention > longest outage; minimal supplemental logging in the CDB root plus `(ALL) COLUMNS` on every captured table (signal and heartbeat too); common user `C##DBZUSER` with `CREATE SESSION`, `SET CONTAINER`, `LOGMINING`, `FLASHBACK ANY TABLE`, `SELECT ANY TRANSACTION`, `SELECT_CATALOG_ROLE`, `EXECUTE ON DBMS_LOGMNR` and the `V_$` log views (`CONTAINER=ALL`).
- PostgreSQL: `wal_level=logical`; `max_replication_slots`/`max_wal_senders` >= connectors; user with `LOGIN REPLICATION`; set `plugin.name=pgoutput` (documented default is `decoderbufs`); one slot + one publication per connector (DBA creates the publication); `REPLICA IDENTITY FULL` where old values are needed.
- MySQL/MariaDB: `binlog_format=ROW`, `binlog_row_image=FULL`, empty `binlog_row_value_options`, binlog retention, GTIDs; unique `database.server.id`; grants `SELECT, RELOAD, SHOW DATABASES, REPLICATION SLAVE, REPLICATION CLIENT` (+ `LOCK TABLES`).
- SQL Server: SQL Server Agent running; `sys.sp_cdc_enable_db` (sysadmin) and `sys.sp_cdc_enable_table` per table (db_owner) with a gating role; a new capture instance after DDL.
- MongoDB: replica set or sharded cluster (no standalone); `mongodb.connection.string`; read on the database; `changeStreamPreAndPostImages` for before images; oplog window > outage.
- Every captured table has a primary key or `message.key.columns`.

3. Minimal bookshop-oracle-source

```
{"name": "bookshop-oracle-source",
 "config": {
  "connector.class":
    "io.debezium.connector.oracle.OracleConnector",
  "tasks.max": "1",
  "database.hostname": "oracle",
  "database.port": "1521",
  "database.user": "c##dbzuser",
  "database.password":
    "${env:BOOKSHOP_ORACLE_PASSWORD}",
  "database.dbname": "FREE",
  "database.pdb.name": "FREEPDB1",
  "topic.prefix": "legacy",
  "database.connection.adapter": "logminer",
  "log.mining.strategy": "hybrid",
  "log.mining.query.filter.mode": "in",
  "table.include.list":
    "BOOKSHOP.BOOK,BOOKSHOP.AUTHOR,BOOKSHOP.CUSTOMER,
    BOOKSHOP.ORDERS,BOOKSHOP.ORDER_LINE,
    BOOKSHOP.SUPPLIER_CATALOGUE,
    BOOKSHOP.DEBEZIUM_HEARTBEAT",
  "snapshot.mode": "initial",
  "schema.history.internal.kafka.bootstrap.servers":
    "kafka:9092",
  "schema.history.internal.kafka.topic":
    "legacy.schema-history",
  "key.converter":
    "org.apache.kafka.connect.json.JsonConverter",
  "key.converter.schemas.enable": "true",
  "value.converter":
    "org.apache.kafka.connect.json.JsonConverter",
  "value.converter.schemas.enable": "true",
  "decimal.handling.mode": "string",
  "provide.transaction.metadata": "true",
  "signal.enabled.channels": "source,kafka",
  "signal.data.collection":
    "FREEPDB1.BOOKSHOP.DEBEZIUM_SIGNAL",
  "signal.kafka.topic": "legacy.signals",
  "signal.kafka.bootstrap.servers": "kafka:9092",
  "notification.enabled.channels": "sink,log",
  "notification.sink.topic.name":
    "legacy.notifications",
  "heartbeat.interval.ms": "10000",
  "heartbeat.action.query":
    "UPDATE BOOKSHOP.DEBEZIUM_HEARTBEAT
    SET LAST_BEAT = SYSTIMESTAMP WHERE ID = 1"}}
```

- Strings broken over lines are one string: join `table.include.list` without spaces, the heartbeat SQL with one space.
- Schemas on: without them the JDBC sink skips every record. Decimals as strings: every `NUMBER`, keys included, arrives as a JSON string.
- Keep `schema.history.internal.store.only.captured.tables.ddl=false` (the default): rollback and reverse replication rely on it.
- The `${env:...}` value needs a config provider on the worker. `topic.prefix` is part of the offset key: never change it.

4. Change event envelope and op codes

- Key: the primary key columns (or `message.key.columns`), e.g. `{"BOOK_ID":"7"}` with `decimal.handling.mode=string`. It picks the partition, so order is per row.
- Value: `before`, `after`, `source`, `op`, `ts_ms`/`ts_us`/`ts_ns` (processing time in the Connect JVM), `transaction` (with `provide.transaction.metadata=true`).
- Oracle `source`: `scn`, `commit_scn`, `txId`, `user_name`, `schema`, `table`, `ts_ms` (time of the change in the database). Capture lag = `ts_ms - source.ts_ms`.
- JSON converter with schemas: `{"schema":{...},"payload":{...}}`. Debezium sends no changed-column list: compare `before` and `after`.

op	Meaning	Notes
c	create (insert)	`before` is null
u	update	both images; a PK update is a d then a c
d	delete	`after` is null, then a tombstone (`tombstones.on.delete=true`)
r	read	snapshot row (initial, incremental or blocking)
t	truncate	key and images null; Oracle skips it (`skipped.operations=t`)
m	message	PostgreSQL `pg_logical_emit_message` only

5. Snapshots and signals

snapshot.mode	Behaviour
`initial` (default)	Schema + data once, then stream
`initial_only`	Snapshot, then stop; never streams
`no_data`	Structure only, stream from now (was `schema_only`)
`when_needed`	Snapshot if no offset or offset no longer in the logs
`always`	Snapshot at every start
`recovery`	Rebuild lost schema history (was `schema_only_recovery`)
`configuration_based`	Five `snapshot.mode.configuration.based.*` flags
`custom`	`Snapshotter` SPI via `snapshot.mode.custom.name`

Incremental snapshot through the signal table (as BOOKSHOP in FREEPDB1):

```
INSERT INTO BOOKSHOP.DEBEZIUM_SIGNAL (ID, TYPE, DATA)
VALUES ('backfill-book-2026-11-09',
  'execute-snapshot',
  '{"data-collections": ["FREEPDB1.BOOKSHOP.BOOK"],
    "type": "incremental"}');
COMMIT;
-- also 'stop-snapshot' (same DATA),
-- 'pause-snapshot' / 'resume-snapshot' (no DATA),
-- "type": "blocking", and 'log' to test the channel
```

Grant missing from the official list (else ORA-41900 when the connector writes its watermarks):

```
GRANT INSERT, DELETE ON BOOKSHOP.DEBEZIUM_SIGNAL
  TO C##DBZUSER;
```

- Table: `ID VARCHAR2(42)` PK, `TYPE VARCHAR2(32) NOT NULL`, `DATA VARCHAR2(2048)`, with `(ALL) COLUMNS` logging. Not needed in `table.include.list`.
- `signal.data.collection` uses the PDB form `FREEPDB1.BOOKSHOP.DEBEZIUM_SIGNAL`; the CDB form `FREE...` is silently ignored. Check with a `log` signal.
- Kafka channel: topic `legacy.signals`, record key `legacy`. Progress arrives on `legacy.notifications` (`STARTED`, `IN_PROGRESS`, `COMPLETED`).

6. Write cut-over checklist

- Before: reads already cut over; cut-over and rollback rehearsed on staging; reverse connectors `bookshop-pg-source` and `bookshop-oracle-reverse-sink` registered `STOPPED`.

```
1 Freeze writes of the capability (flag or proxy)
2 Drain: MilliSecondsBehindSource ~ 0, no lag for
    bookshop-legacy-sync and
    connect-bookshop-pg-mirror-sink
3 Final reconciliation: every chunk matches
4 PATCH the forward config: drop moved tables from
    table.include.list, set
    log.mining.username.exclude.list=BOOKSHOP_SYNC;
    remove their topics from the mirror sink
5 Write guard (trigger) on the legacy tables
6 Resume bookshop-pg-source (creates the slot
    bookshop_reverse), then the reverse sink
7 Route writes to the new app, lift the freeze
8 Smoke test: change arrives in Oracle; no event
    with source.user_name=BOOKSHOP_SYNC on legacy.*
```

- Loop prevention: user exclude on Oracle; replication origins or an origin marker column + Filter on PostgreSQL.
- A stopped reverse connector after a rollback: resume within hours or drop its slot (WAL fills the disk).

7. Renamed or removed in 3.7 (and earlier)

```
database.server.name     -> topic.prefix
database.history.*       -> schema.history.internal.*
snapshot.mode values (old ones: deprecated aliases):
  schema_only            -> no_data
  schema_only_recovery   -> recovery
drop.tombstones, delete.handling.mode (removed)
                  -> delete.tombstone.handling.mode
mongodb.hosts            -> mongodb.connection.string
ByLogicalTableRouter     -> ToLogicalTopicRouter
log.mining.strategy=redo_log_catalog (deprecated)
                         -> hybrid
Oracle continuous mining -> support removed in 3.7
Cassandra 3 connector    -> removed in 3.7
Debezium UI    -> Debezium Management Platform
```

- A renamed property is silently ignored: read the release notes and validate the config before an upgrade.

8. JDBC sink essentials (bookshop-pg-mirror-sink)

```
connector.class=
  io.debezium.connector.jdbc.JdbcSinkConnector
topics=legacy.BOOKSHOP.BOOK,legacy.BOOKSHOP.AUTHOR,
  legacy.BOOKSHOP.SUPPLIER_CATALOGUE
connection.url=jdbc:postgresql://postgres:5432/bookshop
  ?stringtype=unspecified
connection.username=bookshop_mirror
insert.mode=upsert
primary.key.mode=record_key
delete.enabled=true
schema.evolution=none
collection.name.format=legacy_${source.table}
quote.identifiers=false
use.time.zone=UTC
```

- Shown as properties; values split over two lines are one string. In JSON, every value is a string.
- `?stringtype=unspecified` is required: without it PostgreSQL rejects the string keys (bigint but expression is of type character varying).
- `upsert`: idempotent writes under at-least-once delivery. `record_key`: the event key is the primary key. `delete.enabled=true`: deletes and tombstones delete rows (needs a key mode).
- `schema.evolution`: none with pre-created `legacy_*` tables. `validate-only` (new in 3.7) fails at startup with `${source.*}` names. `basic` only creates tables and adds columns.
- DLQ only with `internal.enable.sces=true` (default `false` in 3.7.1) + `errors.tolerance=all` + `errors.deadletterqueue.topic.name=bookshop-pg-mirror-sink.dlq`.
- Consumes native Debezium events: no flattening SMT needed.

9. Key transformations (SMTs)

```
transforms=unwrap
transforms.unwrap.type=
  io.debezium.transforms.ExtractNewRecordState
transforms.unwrap.delete.tombstone.handling.mode=
  rewrite
transforms.unwrap.add.fields=op,table,source.ts_ms
```

- `delete.tombstone.handling.mode`: `tombstone` (default), `drop`, `rewrite` (`__deleted=true`, soft delete), `rewrite-with-tombstone`, `delete-to-tombstone`.
- `io.debezium.transforms.outbox.EventRouter`: `route.by.field=aggregatetype`, `route.topic.replacement=outbox.event.${routedByValue}`, `table.field.event.key=aggregateid`, `table.expand.json.payload=true`.
- `io.debezium.transforms.Filter`: `language=jsr223.groovy`, `condition` true keeps the record. Needs `ENABLE_DEBEZIUM_SCRIPTING=true` and the Groovy 4.0.17 jars (`groovy`, `groovy-jsr223`, `groovy-json`); secure the REST API first.
- `io.debezium.transforms.ToLogicalTopicRouter` (was `ByLogicalTableRouter`): `topic.regex`, `topic.replacement`; adds `__dbz__sourceIdentifier` to the key unless `key.enforce.uniqueness=false`.
- Limit an SMT to some topics with a predicate: `predicates.x.type=...TopicNameMatches`, `pattern`.

10. Kafka Connect REST (http://localhost:8083)

```
POST   /connectors              create (@file.json)
         optional body key "initial_state": "STOPPED"
PUT    /connectors/{n}/config   create or replace
PATCH  /connectors/{n}/config   merge some keys
GET    /connectors/{n}/status   state + task trace
PUT    /connectors/{n}/pause  | /resume
PUT    /connectors/{n}/stop     -> STOPPED
POST   /connectors/{n}/restart?includeTasks=true
         &onlyFailed=true
GET    /connectors/{n}/offsets  read (back up first)
PATCH  /connectors/{n}/offsets  alter (STOPPED only)
DELETE /connectors/{n}/offsets  reset (STOPPED only)
DELETE /connectors/{n}          remove
PUT    /connector-plugins/OracleConnector/config/
         validate               check a config
```

- Offsets can only be changed when the connector is `STOPPED`: `PAUSED` returns 400. `stop` also releases the LogMiner session; `resume` starts it again.
- Back up the offsets (`GET`) before a `PATCH` or `DELETE`. After a reset the connector starts like a new one (snapshot again).
- Restarts replay events (at-least-once): idempotent consumers, or `exactly.once.source.support=enabled` on workers + `exactly.once.support=required`.

11. Metrics to watch

MBeans `debezium.oracle:type=connector-metrics, context=snapshot|streaming|schema-history, server=legacy`

Question	Metric
Running?	Connect task status, `Connected`; alert on missing task metrics
Snapshot progress	`RemainingTableCount`, `RowsScanned` (per table), `SnapshotCompleted`, `SnapshotAborted`
Lag	`MilliSecondsBehindSource` (-1 until first event), `LagFromSourceInMilliseconds`; sink lag = consumer lag
Alive?	`MilliSecondsSinceLastEvent`, heartbeat topic `__debezium-heartbeat.legacy`
Back-pressure	`QueueRemainingCapacity` vs `QueueTotalCapacity`
Long transactions	`OldestScnAgeInMilliseconds`, `NumberOfActiveTransactions`, `AbandonedTransactionCount`
Errors	`NumberOfErroneousEvents`, `TotalSchemaChangeParseErrorCount`
Safe restart?	`OffsetScn` vs oldest archive log; schema history `Status`
PostgreSQL	WAL retained by each slot in `pg_replication_slots`